

E-Entity Relationship

Prev

- ER
- Attribute
- Cardinalitas
- Constraint

Step By Step Design ER Model

1. Define scope **Miniworld**
2. Data **needs analysis**
3. Define **entity**
4. Define strong/weak entity
5. Define attributes each entity
6. Define **Relationships**
7. Define **cardinality and constraints**

Next Lesson



ER with an Object-Oriented approach

Definition

- The resulting model is called the enhanced-ER or Extended ER (E2R or EER) model
- Includes **all modeling concepts of basic ER**
- **Additional concepts**: subclasses/superclasses, specialization/generalization, categories, attribute inheritance
- It is used to model applications **more completely and accurately if needed**
- It includes **some object-oriented concepts**, such as **inheritance**

Why Need E-ER

- Sometimes, ER is not enough to express the semantic abstraction of database problem of miniworld
- A few concepts in OO are added in ER, then named EER
 - ❖ Superclass/Subclass Relationship
 - ❖ Specialization/Generalization
 - ❖ Categorization

Superclass - Subclass

- Superclass: An entity whose attributes are used by another entity (its attributes are inherited)
- Subclass: An entity using attribute another entity.
- Entity in a subclass must be a member of a superclass, but not vice-versa!
- **Note: When subclass is needed**
 - ❖ An entity type may have additional meaningful subgrouping of its populated data
 - ❖ May have special attributes (Specialization)
 - ❖ May have special relationship

Sample Superclass - Subclass

- EMPLOYEE (based on activity)
 - Secretary → TypingSpeed
 - Engineer → EngType
 - Technician → TechGrade
- EMPLOYEE (based on salary)
 - SALARIED_EMPLOYEE
 - HOURLY_EMPLOYEE
- Employee can be sub-grouped into:
 - Secretary, Engineer, Technician → subclass
- EMPLOYEE entity type is the superclass

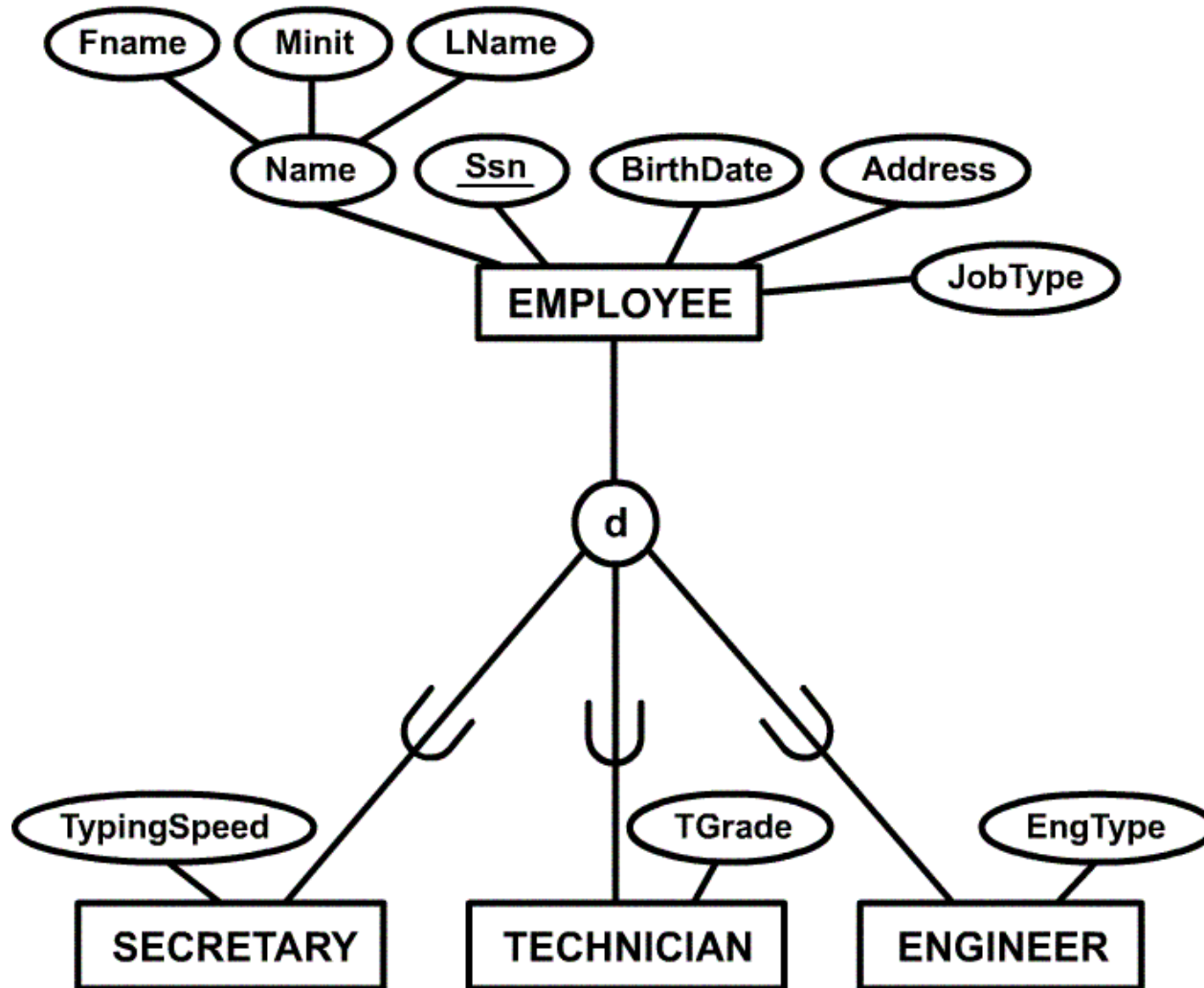
Inheritance

- Notation Subset symbol $\subset$
- An entity that is member of a **subclass** *inherits* all **attributes** of the entity as a member of the **superclass**
- It also inherits **all relationships**
- So, We will get to know the terms:
 - a. **Specialization**
 - b. **Generalization**

Specialization

- The process of defining subclasses from a superclass, while generalization is the process of defining a superclass from two or more subclasses.
- Example:
Employee can be sub-grouped into:
 - Secretary, Engineer, Technician → subclass

Specialization (sample)



Generalization

- The process of defining a superclass from two or more subclasses.
- Example: truck & car can be generalized into **VEHICLE**
- Inverse of the specialization process

Generalization (sample)

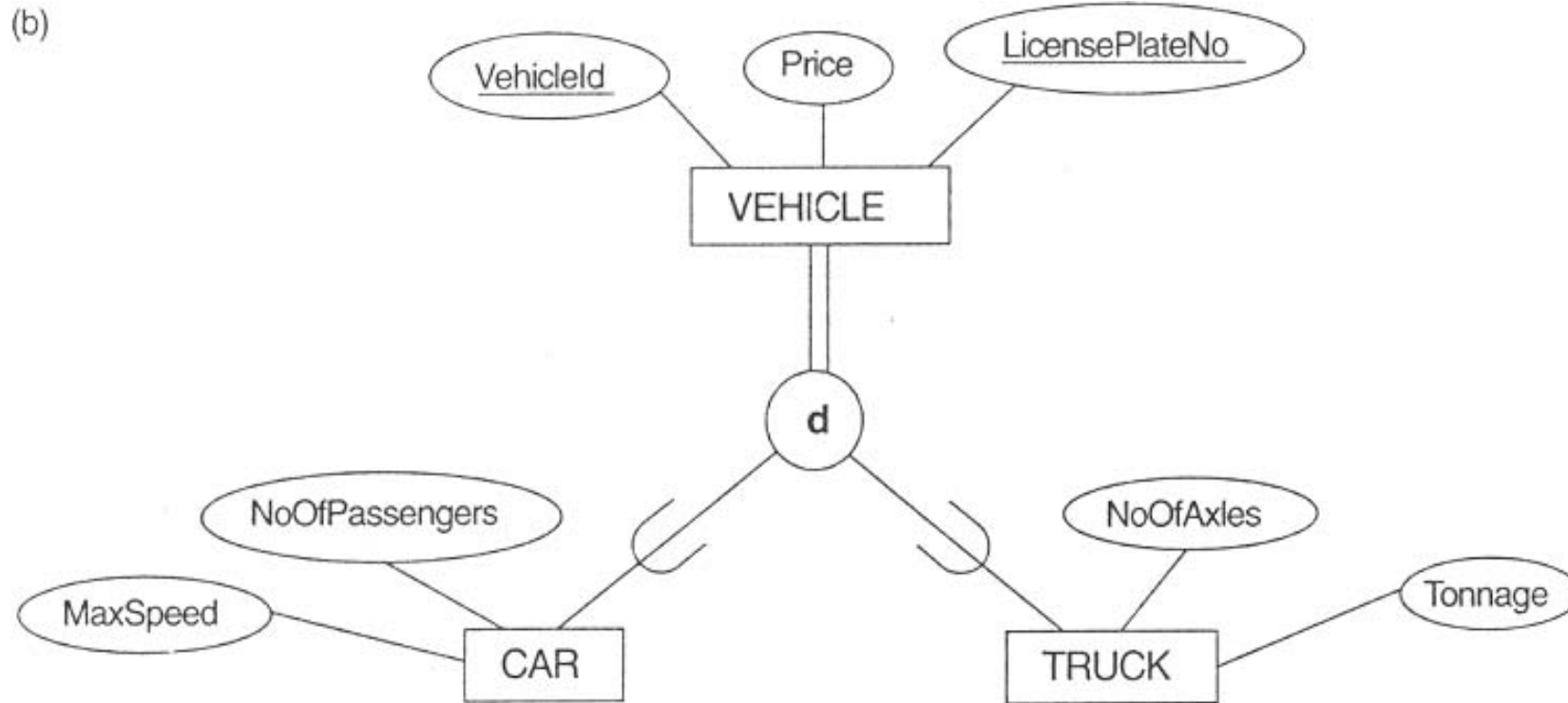


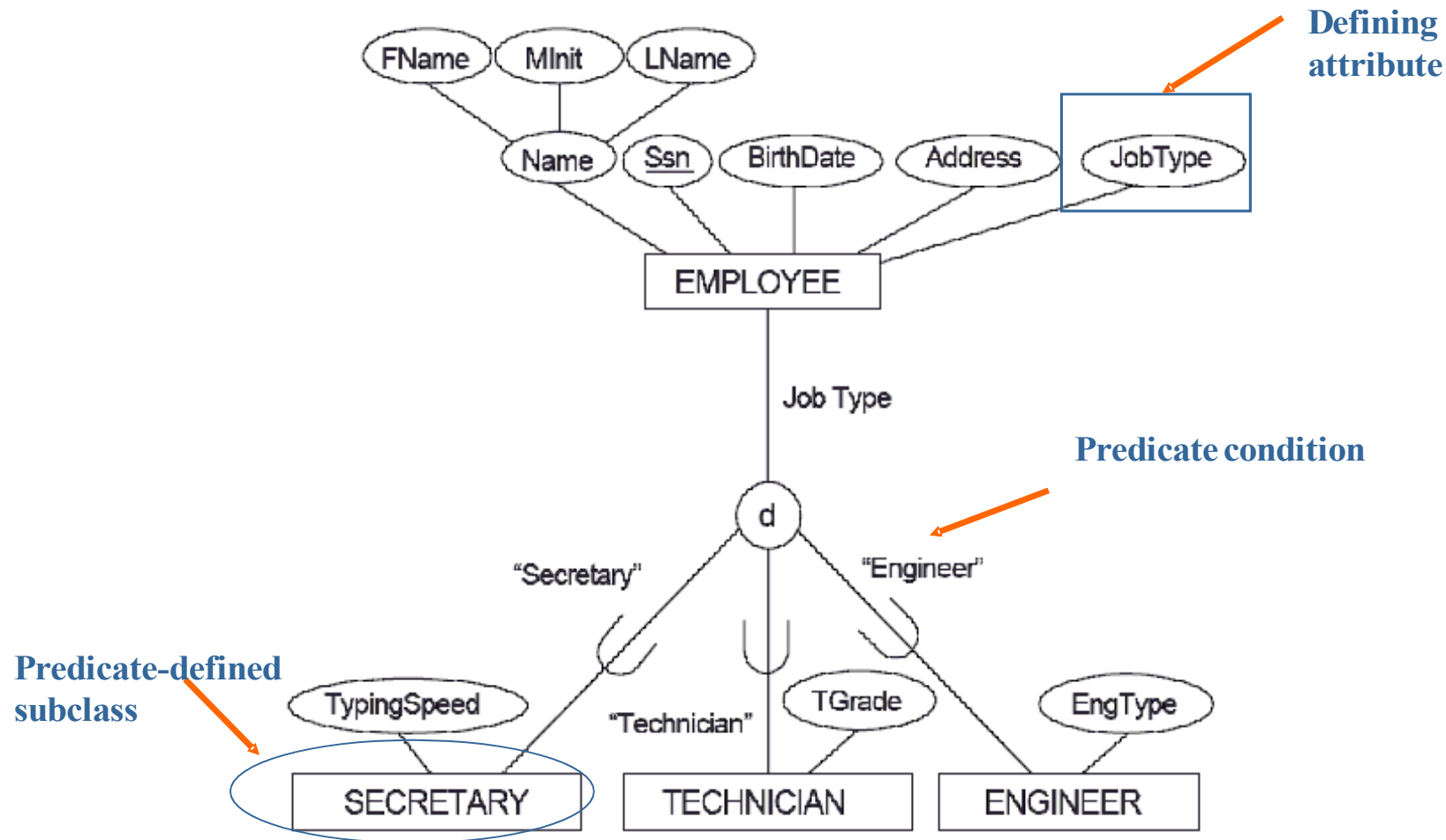
Figure 4.3 Examples of generalization. (a) Two entity types CAR and TRUCK.
(b) Generalizing CAR and TRUCK into VEHICLE.

Constraints on Specialization and Generalization

- Attribute-defined specialization
 - Based on values of a superclass attribute (defining attribute)
 - All subclasses have their member condition on the same attribute of the superclass
 - Predicate-defined (condition defined) subclass
 - JobType = 'Engineer' => defining predicate
- User-defined Subclass
 - Each membership is determined by the user

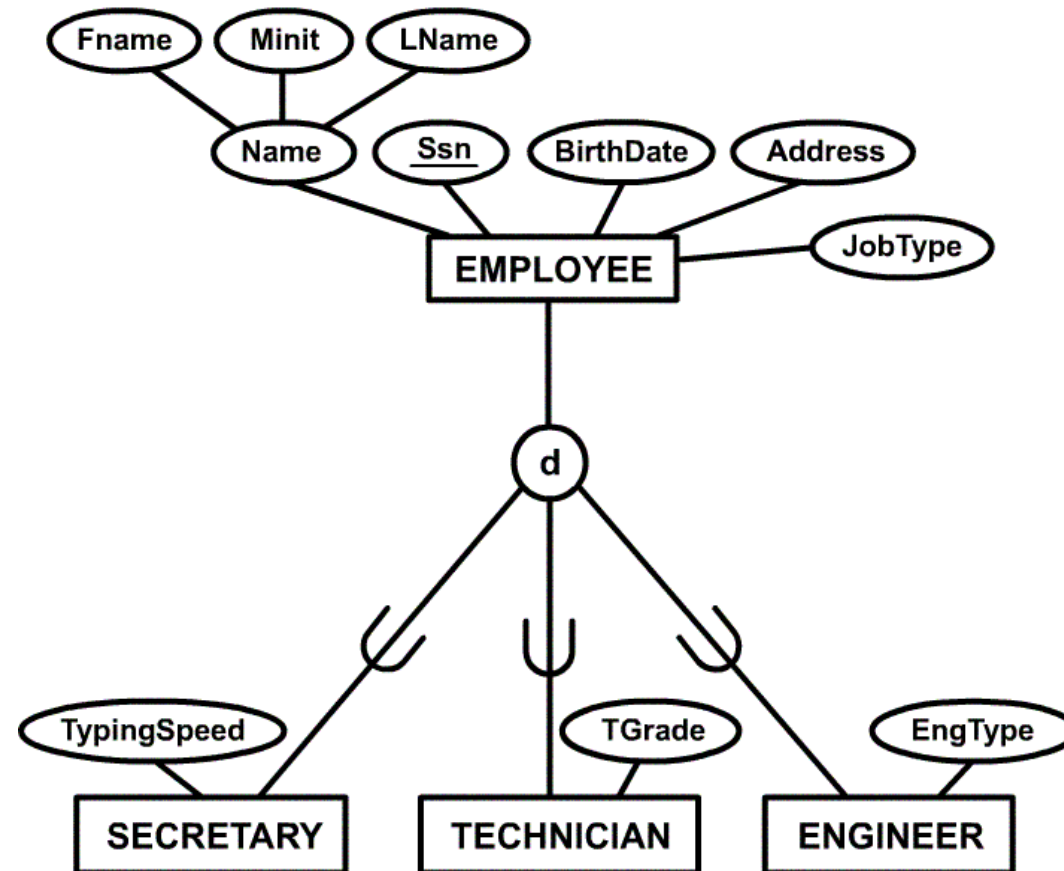
Constraints on Specialization and Generalization

Figure 4.4 An attribute-defined specialization on the JobType attribute of EMPLOYEE.



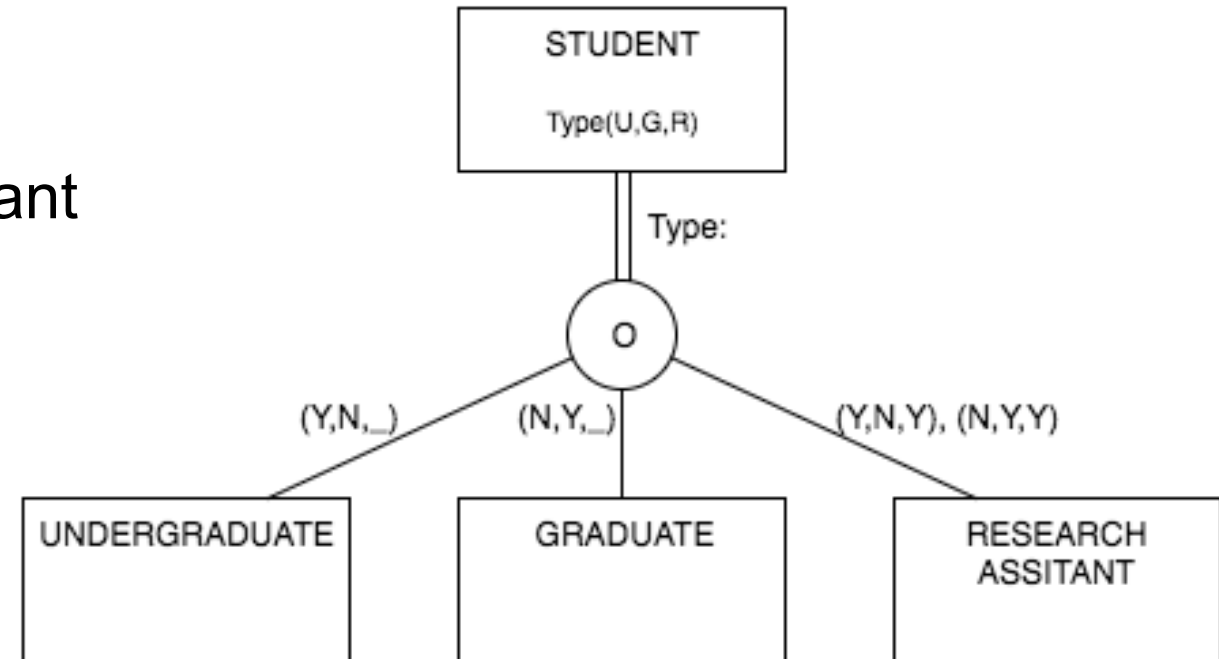
Disjoint

- An entity **can only be at most one** of the subclass
- Look at the previous EER diagram
- Use (**d**)



Overlap

- The **same entity** may be a **member of more than one subclass** of the specialization
- Use the (o)
- Example, a student can be:
 - Undergraduate & Research Assistant
 - Graduate & Research Assistant
 - Undergraduate
 - Graduate
 - Research Assistant



Completeness Constrains

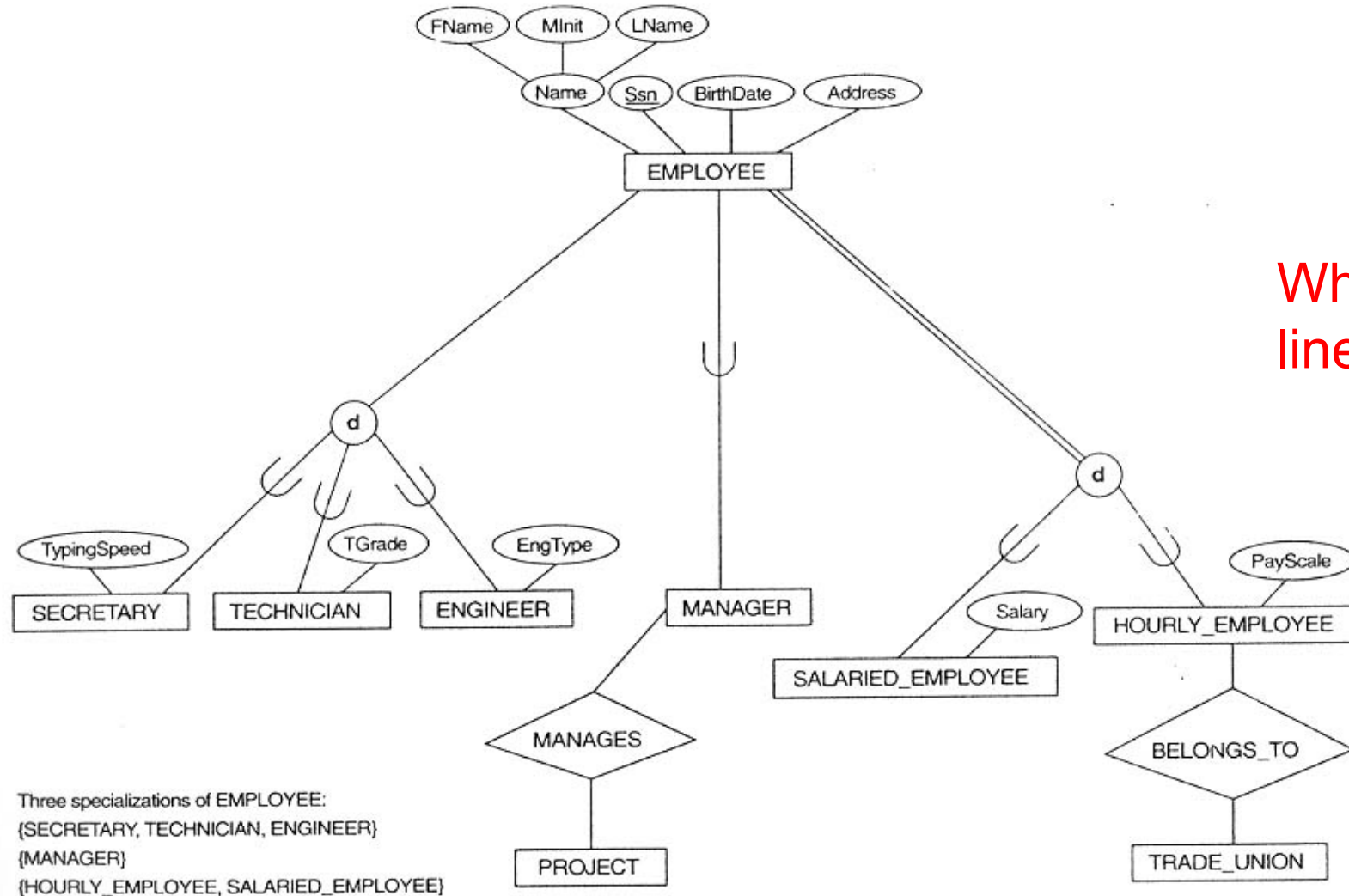
- **Total:**

- Every **entity** in the superclass **must be** a member of some subclass
- Example Employee, the Salaried_Employee and Hourly_Employee
- Shown using double line

- **Partial:**

- Allows an entity **not to belong to any** subclass
- Example:
Employee contains:
 - Manager
 - Job type (Secretary, Tech, Engineer)
 - Just general employee
- Use single line

Specialization (sample)

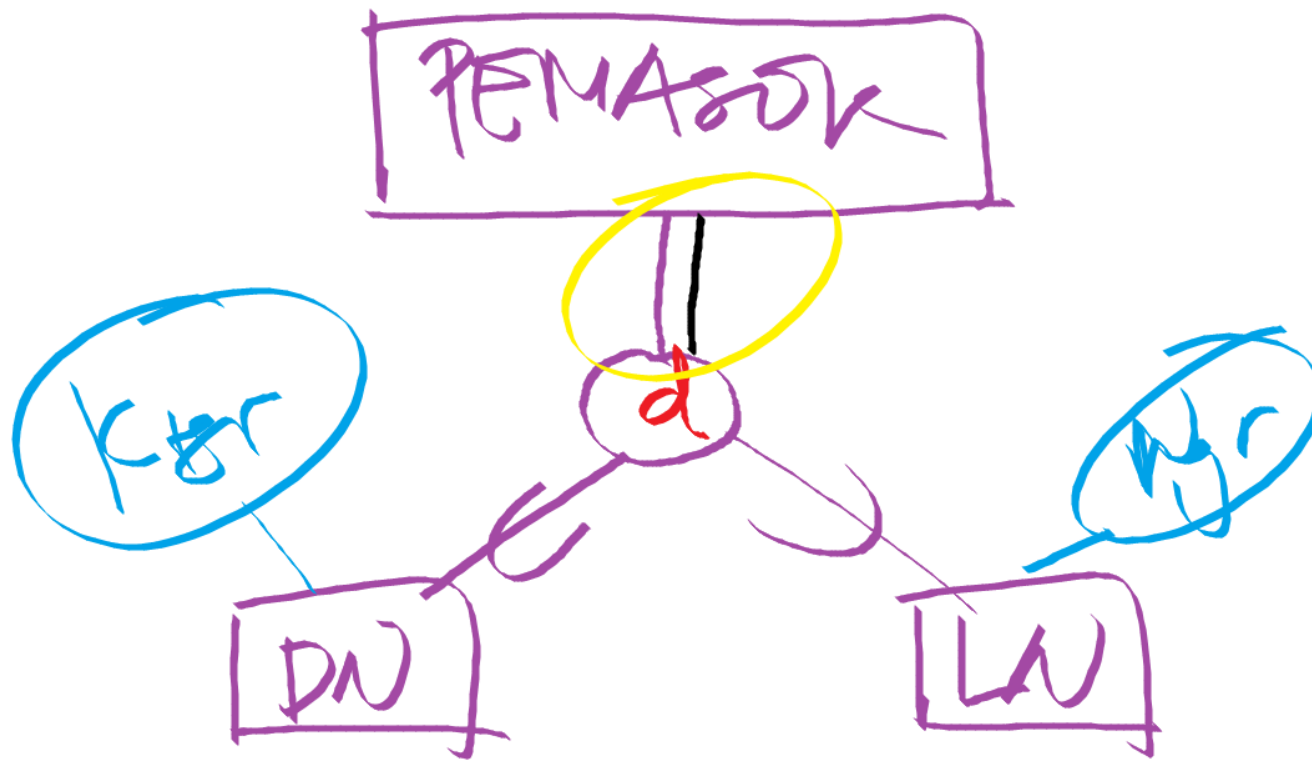


What is different single/double line, See?

Three specializations of EMPLOYEE:
{SECRETARY, TECHNICIAN, ENGINEER}
{MANAGER}
{HOURLY_EMPLOYEE, SALARIED_EMPLOYEE}

Figure 4.1 EER diagram notation for representing specialization and subclasses.

Specialization (sample)



Why double line

Rules

- Deleting entity from a superclass → deletes it also from the subclasses
- Inserting in a superclass, when attribute defined is filled → must insert to the proper subclass as well
- Inserting in superclass of total specialization → must insert into at least one subclass

Hierarchy and Lattice

- **Hierarchy:** a **subclass** only **participates in one** superclass relationship. A superclass can **participate in more than one** subclass relationship.
 - Example: an Engineering, must be an Employee
- **Lattice:** a **subclass** can **participate in more than one** superclass relationship
 - Example: an Engineering Manager, must be an Engineer, and also a Manager!

Hierarchy and Lattice

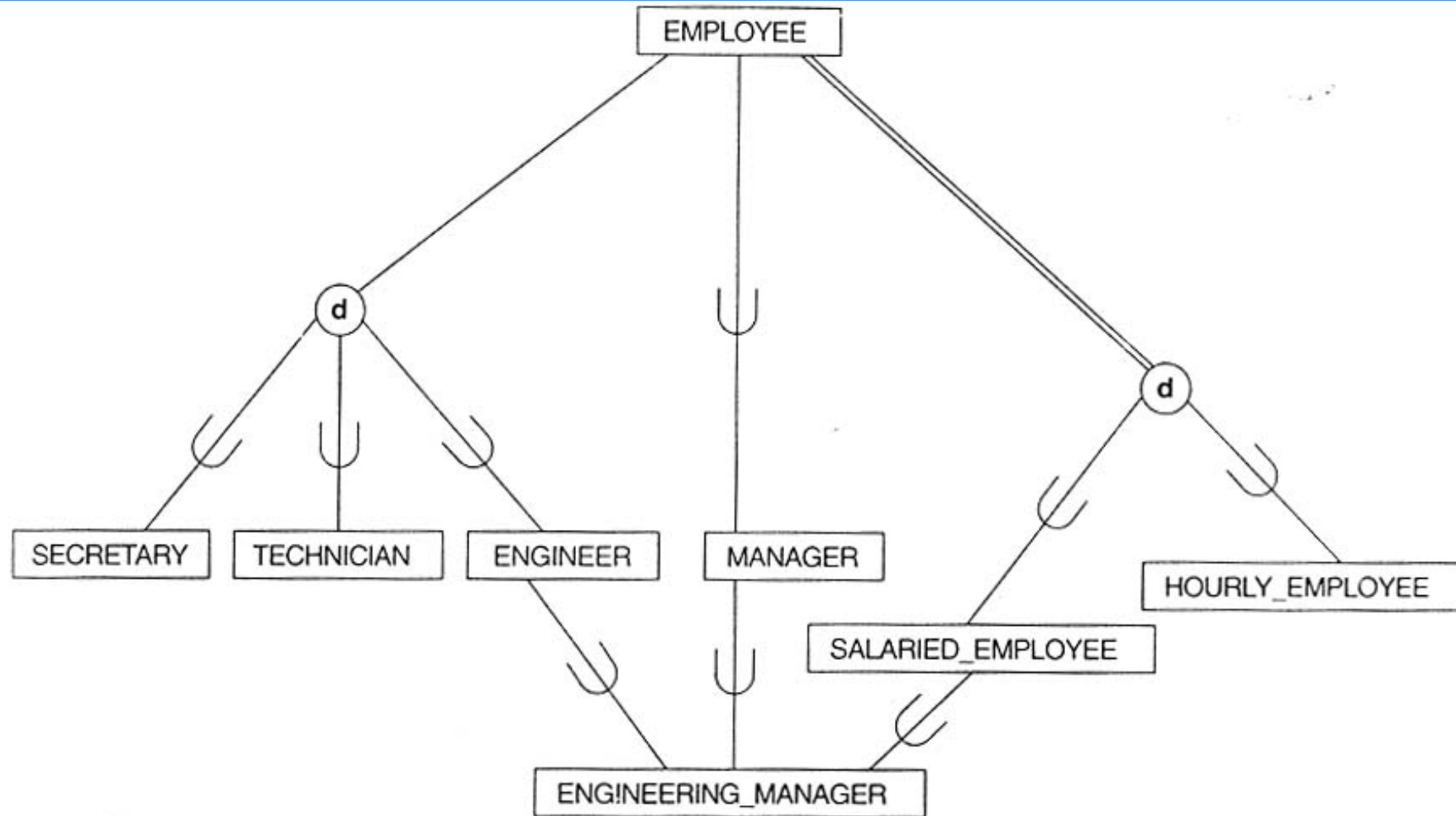
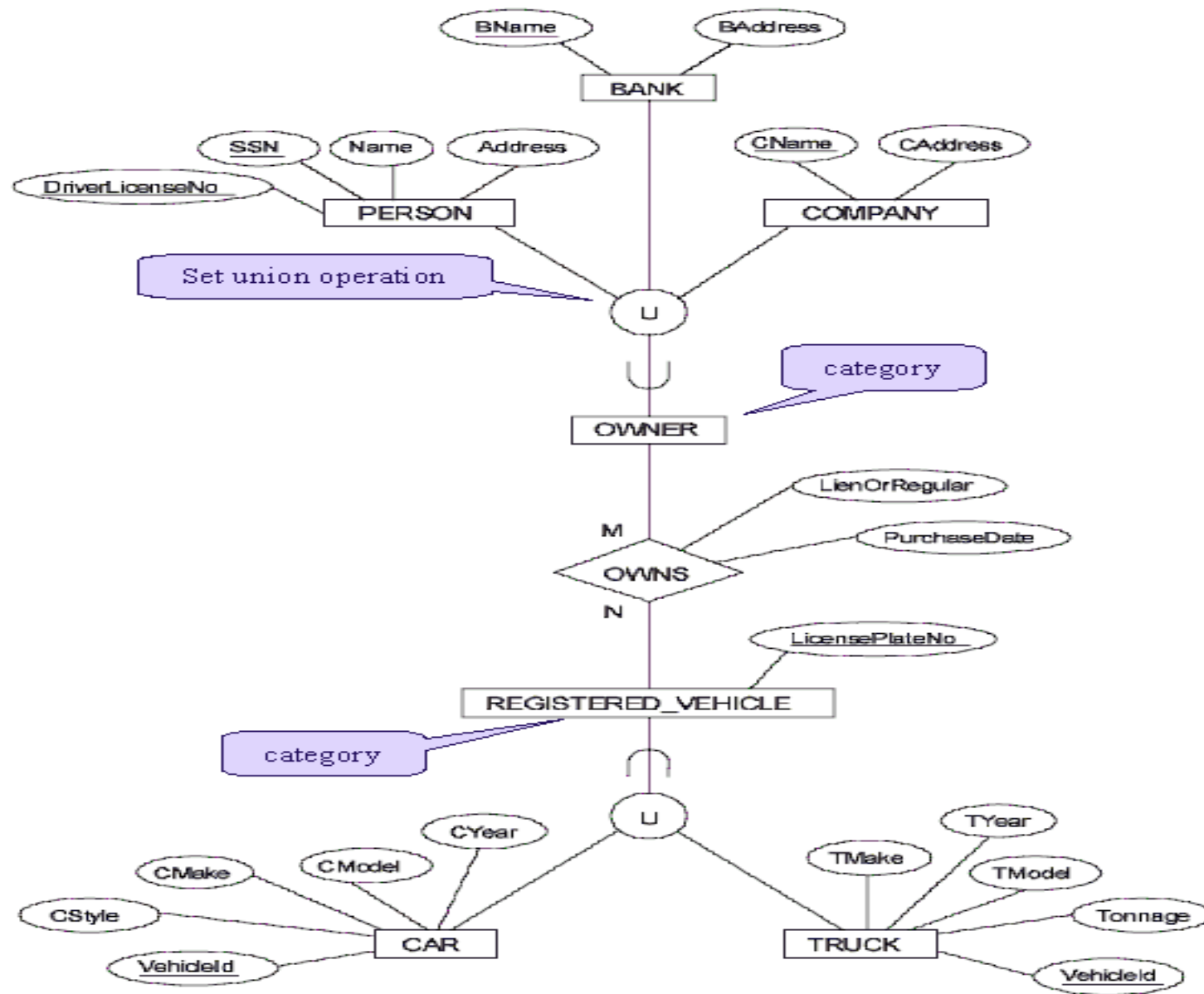


Figure 4.6 A specialization lattice with the shared subclass ENGINEERING_MANAGER.

Union Types Using Categories

- The **subclass represent collection of objects**, which we call **union type** or **category**
- Use the (**U**) symbol
- A **category OWNER is a union** subclass of COMPANY, BANK and PERSON
- **Registered_Vehicle is a union** subclass of Car & Truck

Union Types Using Categories



Recomendation E-ER

- EER model is especially useful is domain being model is OO in nature and use inheritance **reduce the complexity** of the design
- Cases using EER:
 - When using attribute inheritance can reduce the use of nulls in a single entity relation (that contains multiple subclasses)
 - Subclasses can be used to explicitly model and name subsets of entity types that participate in their own relationships

Attention

- Most database projects do not need the object-oriented model features in EER
- Goal of conceptual data modeling is to produce a model that simple and easy to understand
- **Do not use complicated class/subclass relationship if they are not needed**
- Offer significant advantage over regular ER model

Summary

- Superclass/subclass
- Specialization/Generalization
- Disjoint/Overlapping
- Completeness Constrains (single/double line)
- Rules
- Hierarchy and Lattice
- When using

Task

- Show your assignments last week



Barangsiapa yang berangkat menimba ilmu untuk mengamalkan ilmu,
niscaya ilmu yang sedikit pun akan bermanfaat baginya